

Automation Testing with Selenium

Introduction

Selenium is the **most widely-used open-source automation tool for testing web applications**.

It automates browser actions to verify:

- ✓ UI workflows
- ✓ Functional correctness
- ✓ Regression behaviour

Automation helps **reduce manual effort, prevent human error, and increase repeatability**.

1. Introduction to Selenium

1.1 What is Selenium?

Selenium is a **framework of automation tools** used to automate web browsers.

Key Characteristics

- Free and open-source
- Supports almost all browsers — Chrome, Firefox, Edge, Safari
- Works on major operating systems — Windows, macOS, Linux
- Supports many programming languages — Java, Python, C#, JavaScript, Ruby, etc.
- Integrates well with CI/CD pipelines (e.g., Jenkins, GitHub Actions)

Important:

Selenium automates **only web applications**, not desktop or standalone apps.

1.2 Selenium Components

Component	Purpose / Usage
Selenium WebDriver	Core API used for browser automation. This is what most automation engineers use.
Selenium IDE	A record-and-playback tool used for quick prototyping and learning.
Selenium Grid	Used to run automated tests across multiple machines or browsers in parallel.

In real-world projects — WebDriver is the main tool.

2. Selenium WebDriver Setup

To use Selenium WebDriver, you need the following:

Setup Requirements

1. A programming language runtime (Java / Python / C# etc.)
2. Selenium WebDriver libraries
3. A browser driver (e.g., ChromeDriver for Chrome)
4. An IDE (e.g., Eclipse, IntelliJ, VS Code)

Basic Setup Process

1. Install the programming language
2. Add Selenium libraries to your project
3. Download the browser driver
4. Configure project & environment
5. Write the automation script
6. Execute the script — which controls the browser

Concept:

WebDriver sends commands → Browser driver receives → Browser executes action

3. Selenium Commands (Selenese)

Selenese = **commands that automate browser actions.**

They include:

- Opening a website
- Clicking buttons & links
- Typing into fields
- Selecting dropdown options
- Verifying results & outputs

These commands help **simulate real-world user interactions.**

4. Locators — Finding Web Elements

Locators tell Selenium **which element on the page to interact with.**

Accurate locators = **stable & reliable automation.**

Common Selenium Locators

Locator	Meaning
ID	Finds elements by HTML id attribute
Name	Matches name attribute
Class Name	Selects elements by CSS class
Tag Name	Matches HTML tag like <input>
Link Text	Clicks link using full visible text
Partial Link Text	Matches part of a link text
XPath	Selects elements by navigating HTML structure
CSS Selector	Uses CSS rules to locate elements

XPath — Key Concept

XPath is powerful for finding **dynamic or complex elements**.

Types of XPath

- **Absolute XPath** — full path from root (fragile, not recommended)
- **Relative XPath** — shorter & more stable (recommended)

XPath can select elements using:

- attributes
 - text
 - index
 - parent-child relationships
-

5. UI Element Interactions

Selenium can automate **almost all user interactions on a web page**.

These interactions simulate real user actions to validate system functionality.

5.1 Text Boxes

Automation actions include:

- Entering (typing) text
- Clearing existing text
- Validating that the text was entered correctly

Common usage examples:

- Login forms
 - Search bars
 - Registration forms
 - Data-entry fields
-

5.2 Radio Buttons

Radio buttons allow the user to **select only one option from a group**.

Automation should verify:

- The default selected option
 - That selection can be changed
 - Read-only or disabled behaviour
-



5.3 Checkboxes

Checkboxes allow **one or more options to be selected**.

Automation ensures:

- ✓ Correct toggle behaviour (select / unselect)
 - ✓ Default checked / unchecked state
 - ✓ Multi-selection works properly
-

5.4 Dropdowns

There are two types:

- **Single-select dropdown** — only one value may be chosen
- **Multi-select dropdown** — multiple values may be chosen

Automation actions include selecting items:

- By visible text
 - By value
 - By index
-

5.5 Keyboard & Mouse Actions

Some applications require complex user interactions. Selenium supports:

- Mouse hover
- Right-click
- Double-click
- Drag-and-drop
- Keyboard shortcuts (Enter, Tab, Ctrl, etc.)

These actions are automated using the **WebDriver Actions API**.

5.6 Multi-Select Elements

Automation validates:

- Selecting multiple choices
- Removing / deselecting items
- Behaviour when combining selections

Common examples include filters, preference selection, and advanced forms.

6. Test Automation Strategy

Automation with Selenium is **most effective for:**

- ✓ Regression testing
- ✓ Repeated execution

- ✓ Multi-browser validation
 - ✓ Data-driven testing
 - ✓ Smoke testing
-

Automation is NOT ideal for:

- Unstable or highly dynamic UI elements
- Pure visual validation (without additional tools)
- One-time or rarely-used tests

Golden Rule:

Automation should **support manual testing — not replace it.**

7. Creating Test Suites

A **test suite** is a logical group of automated test cases that execute together.

Good test suite design includes:

- Independent test cases
- Reusable functions
- Setup & teardown scripts
- Reporting
- Exception handling
- Logging

Common Automation Frameworks

Java: TestNG, JUnit

Python: PyTest, unittest

Suites enable:

- ✓ Batch execution
 - ✓ CI/CD pipeline integration
 - ✓ Scheduled execution
 - ✓ Parallel test execution
-

8. Benefits of Selenium Automation

Selenium provides:

- ✓ Faster execution compared to manual testing
- ✓ Repeatable and reliable test runs
- ✓ Cross-browser testing capability
- ✓ High ROI for regression testing
- ✓ Easy integration with DevOps pipelines
- ✓ Team collaboration support
- ✓ Compatibility with cloud execution platforms



Cloud tools such as **BrowserStack** and **Sauce Labs** enable remote testing across devices and browsers.

9. Limitations of Selenium

Selenium **cannot automate:**

- ✗ Desktop applications
- ✗ Mobile applications (unless using Appium)
- ✗ Performance testing
- ✗ Direct API or database testing

Also requires:

- Skilled scripting knowledge
 - Proper test framework setup
 - Stable element locators
 - Regular maintenance effort
-

